COS 214 Practical 6

Task 1 : TIVIDY Research

We understand a workflow as a model for representing recurring work processes as a structured sequence of tasks that transform an input into an outcome. A Workflow Management System (WfMS) provides the capabilities for modeling, instantiating, executing, and monitoring these workflows. It describes what work must be performed, who or what performs it, in what order it occurs, under what conditions it may continue, and what happens when work succeeds, fails, is rejected, or requires escalation.

Workflow Definitions and Running Instances

SAP's documentation states that, " a workflow definition specifies which actions should be performed, when, and under what circumstances, while a workflow instance is the actual execution of those actions". A single definition can produce multiple independent instances, each with its own data, participants, and progress. Oracle's APEX documentation says, the structure is determined by the Workflow Definition, and every time a workflow starts, a Workflow Instance is created as a specific runtime from that definition. This is essential as it lets us know if the process design is correct and whether a particular workflow instance is progressing correctly.

Representing and Routing Work Items

Work items are the basic units of execution within a workflow. Routing rules are the logic that determines where a work item goes next and who receives it after each step in a workflow. In systems like IBM TRIRIGA, work is organised into hierarchical stages with queues, each representing a phase with its own responsibilities. The key insight is that routing rules must be configurable, not hard-coded, since organisations assign work by different criteria, role, workload, or data-driven rules. Assignment should therefore reference an abstract rule, resolved to participants at runtime, so changing strategy does not require rewriting work-item classes.

Approvals, Rejection, and Escalation

Approval is one of the most common workflow patterns: a work item is sent to an approver who may approve or reject. Microsoft Power Automate's Approvals connector illustrates this, route work items to reviewers, capture decisions, and automatically take the next action. Response types include Approve or Reject. A 2024 patent describes a controller enabling participants to modify their tasks within a flexible framework, deferring task definition to responsible users rather than requiring full specification upfront, supporting dynamic workflows. Escalation handles non-response: Enate describes how external API actions enter a "Waiting" state and, on timeout, escalate to a human queue based on allocation rules.

External Integration and Failure Handling

A WfMS must interact with external systems, and failure handling must be top priority. IBM's webMethods OneData supports external workflow support, passing pending transactions to an external system via iHooks .External approvers can approve, reject, delegate, or save changes. The Enate uses failure handling, timeouts and escalation. TIVIDY should support external integrations, and have flexible ways to handle failures and recover problems.

History, Audit, and Monitoring

The system maintains history to support auditing and monitoring. Microsoft Entra ID Governance records every workflow processing event in audit logs, while Power Automate analyses history to identify bottlenecks and improve performance. HelloID records details such as what and when it happened and who was involved. For TIVIDY, this highlights the importance of maintaining accurate records linked to the specific workflow definition version used, ensuring that past activities can be audited reliably.

Engine Reusability and Definition Changes

A workflow management system consists of different components working together to schedule tasks and handle data. The 2025 terminology paper highlights that the coordination of these components ensures workflow efficiency.Reusability comes from separating configurable decisions from the engine. The 2024 patent stated that users who are assigned to a specific task can define them if they want,which allows the workflow to develop as requirements change.States should be clearly represented in the system so that the WfMS can track the progress and react appropriately to situations.

Key Implications for TIVIDY's Design

Based on the research, TIVIDY should follow several important design principles. 1. workflow definitions must be separated from running instances so that changes to a definition do not unintentionally affect existing instances. 2. we must arrange routing and approval rules while role mappings, approval chains, and escalation thresholds are treated as data. 3.the system must manage the lifecycles of workflow instances and individual activities.It should also support external system integration and provide reliable failure handling. 4. Failure handling must be a priority,external system integration. 5. Auditing and monitoring should be built into the system to ensure that actions, decisions and state changes can be tracked.

Conclusion: This research explored how workflow management systems handle tasks, approvals, routing, failures and monitoring. These findings will guide TIVIDY's design and pattern selection. To make the system reusable, TIVIDY should allow important rules and processes to be configured rather than hard-coded.

References

1. Ferreira da Silva, R., et al. (2025). A terminology for scientific workflow systems. *Future Generation Computer Systems*.
2. SAP. Workflow Definition versus Workflow Instance. SAP Help Portal.
3. Nimblework. (2024). Workflow in Workitem.
4. Microsoft. (2025). Introduction to approval flows. Power Automate Training.
5. US Patent 20240005242A1. (2024). Dynamic federated workflow programming management.
6. IBM. (2025). Using External Workflow in webMethods OneData. IBM Documentation.
7. Microsoft. (2024). Auditing Lifecycle Workflows . Microsoft Entra ID Governance.
8. Enate. (2025). Trigger External API Actions. Enate Help.
9. Tools4ever. (2024). Service Automation – Auditing.
10. Microsoft. (2025). Monitor your flows. Power Automate Guidance.
11. Oracle. (2025). About Workflows. Oracle APEX End User's Guide.

Task 2

Scenario 1: Material Procurement Request

Organisation and Process

We are modelling a material procurement request process for a construction company. Employees, such as site supervisors and project managers, can submit requests for materials needed on construction sites. The request captures the user's personal details, material description, quantity required, estimated cost, project details and reason for the request. Once submitted, the request is routed to the relevant handlers for review and approval. The final decision, either approval or rejection, is communicated back to the requester.

Major Workflow Stages

1. Request Initiation: The user starts a new material procurement application.
2. Information Collection: The system collects the user's details, requested materials, quantities, estimated costs and project information.
3. Request Submission: The user reviews and submits the application.
4. Request Review: The chain of responsibility is then used to handle different aspects of the application.
5. Financial Approval: The finance manager verifies whether sufficient funds are available, where necessary.
6. Final Decision: The request is approved or rejected.
7. User Notification: The system communicates the decision and any relevant feedback to the requester.

Routing Decisions and Approvals

If the site supervisor finds the request unnecessary, it is rejected. If information is incomplete or incorrect, it is returned to the requester for correction. Requests exceeding the project budget are escalated to the finance manager. If all required handlers approve the request, it proceeds to final approval. Any rejection results in the requester being notified of the reason.

Dependencies, Failures and Outcomes

A request cannot be submitted without all mandatory information. Each review stage must be completed before the request proceeds to the next handler. If a handler delays the review beyond the allocated time, the system escalates the request to the relevant manager. If materials are unavailable, the procurement officer can suggest alternatives.

The final outcome is an approved or rejected procurement request, with the decision communicated to the requester. All submissions, reviews, approvals and rejections are recorded in an activity history. The system can also integrate with inventory management, accounting and email notification systems.

Scenario 2: University application

Organisation and Process

We are modelling a university application process in which prospective students apply for admission to a university programme. Applicants provide their personal details, academic results, preferred qualification and supporting documents. Once submitted, the application is reviewed by the relevant application handlers. The final decision, either acceptance or rejection, is communicated to the applicant.

Major Workflow Stages

1. Application Initiation: The applicant starts a new university application.

2. Information Collection: The system collects personal details, academic results, programme choices and supporting documents.
3. Application Submission: The applicant reviews and submits the application.
4. Initial Review: The admissions department checks that all required information and documents have been provided.
5. Academic Assessment: The relevant handlers evaluate the applicant's academic results against the programme's admission requirements.
6. Admission Decision: The university decides whether to accept or reject the application.
7. Applicant Notification: The system informs the applicant of the outcome.

Routing Decisions and Approvals

If documents are missing, the application is returned to the applicant for completion. If the applicant does not meet the programme's admission requirements, the application is rejected. If the requirements are met, the application proceeds to the final admission decision. Applications requiring additional assessment are forwarded to the relevant faculty or admissions committee.

Dependencies, Failures and Outcomes

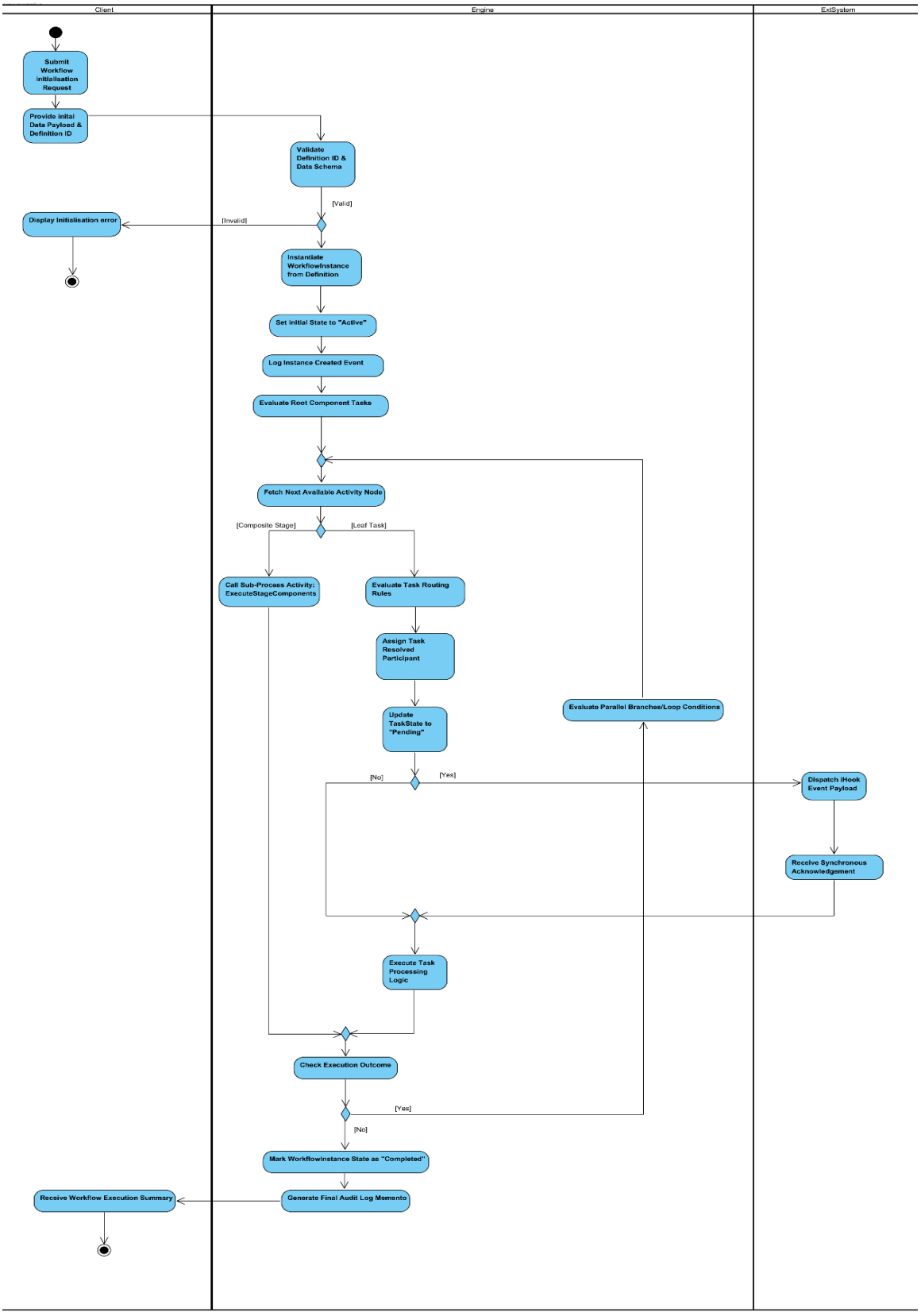
An application cannot be submitted without the mandatory information and documents. Academic assessment can only begin after the initial review is complete. If an application is delayed beyond the allocated processing time, it is escalated to the admissions manager. All submissions, assessments and decisions are recorded in an activity history.

Final Outcome

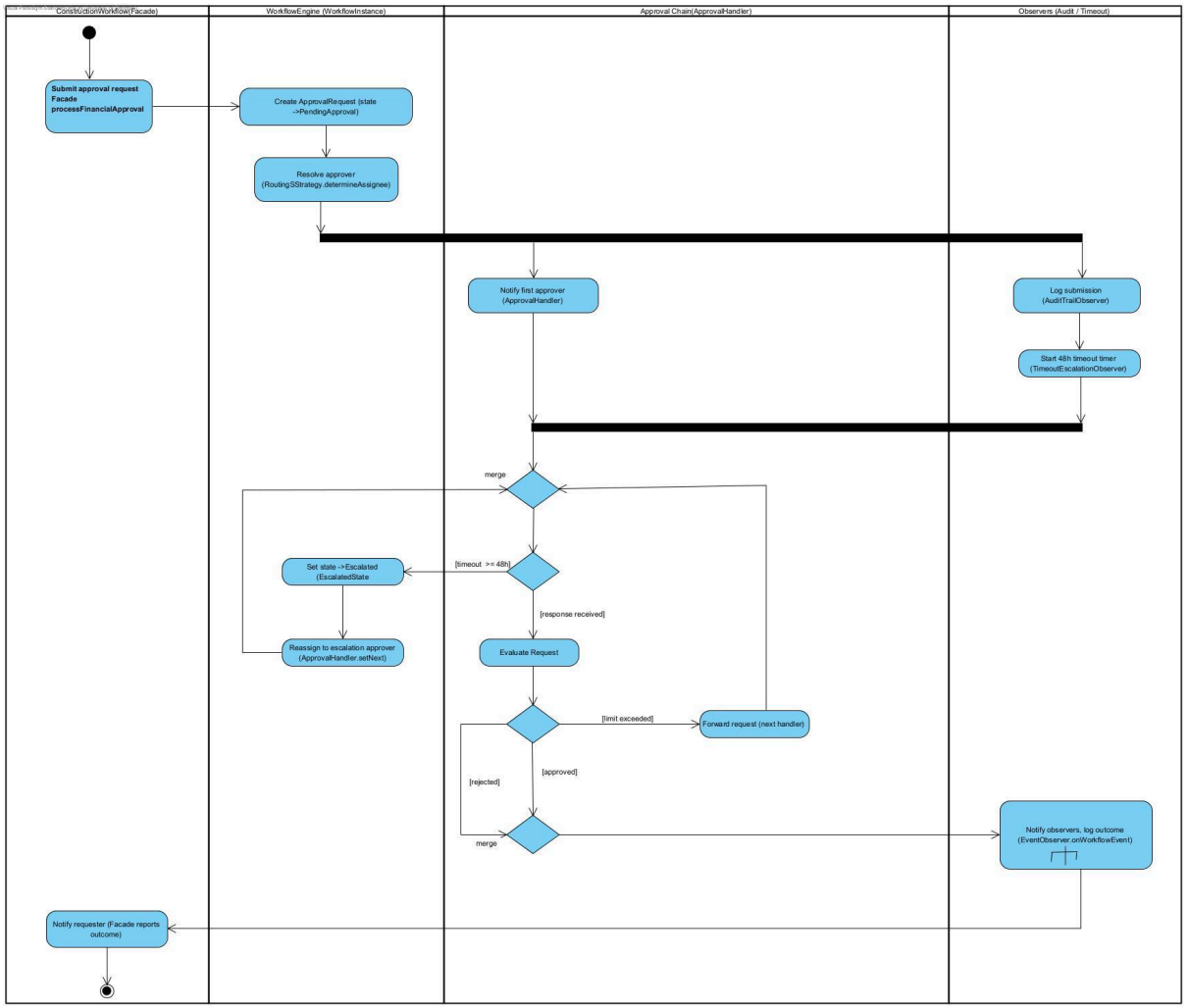
If the application is approved, the applicant receives an offer of admission and instructions on how to accept it and complete registration. If the application is rejected, the applicant receives notification of the decision and, where applicable, the reason for rejection. The workflow ends once the decision has been communicated and recorded. The final expected outcome is a completed admission decision, with successful applicants able to proceed towards university registration.

Task 3: UML Activity Diagrams

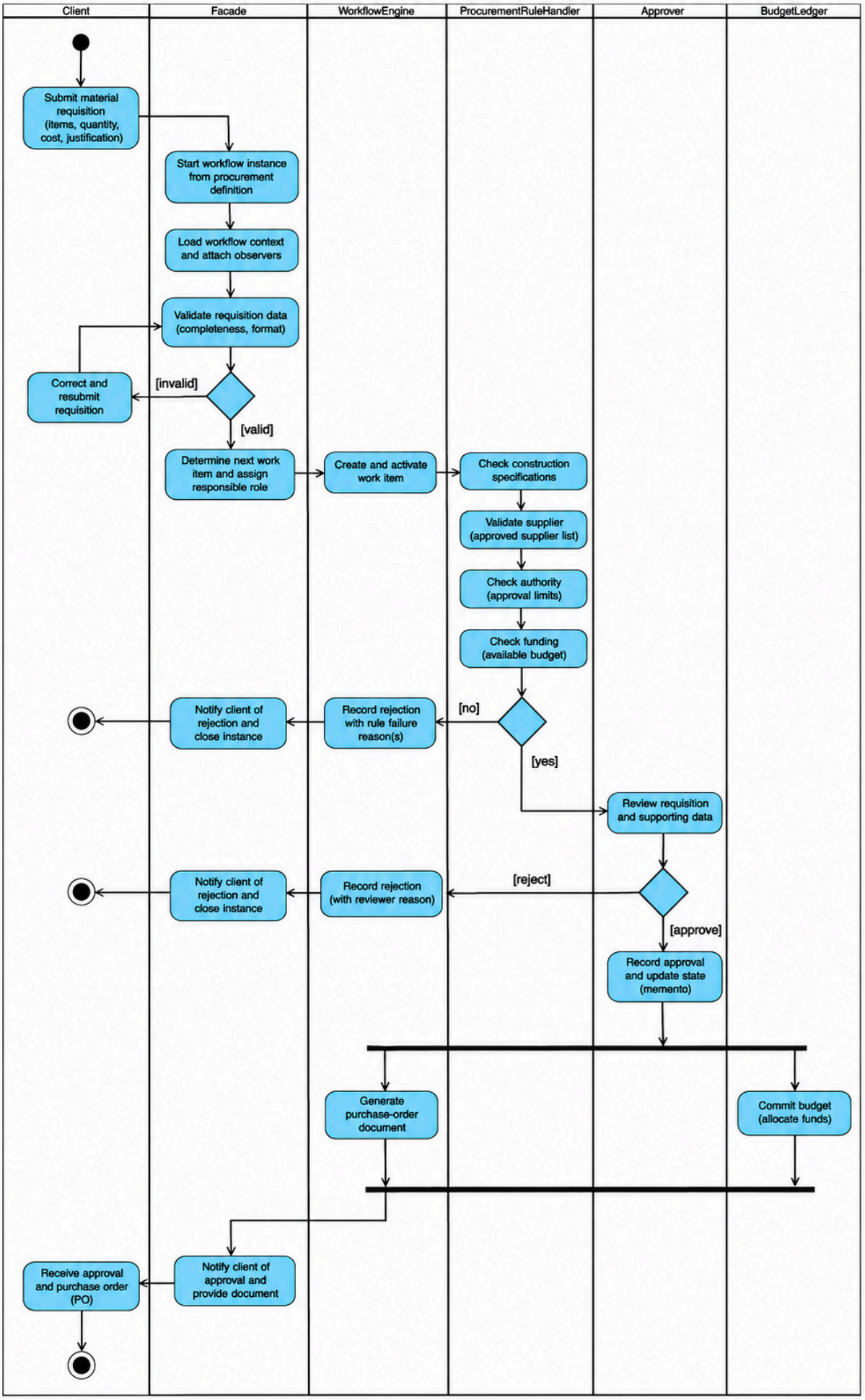
1.



2.


ConstitutionWorkflow(Facade)
WorkflowEngine (WorkflowInstance)
Approval Chain(ApprovalHandler)
Observers (Audit / Timeout)
Submit approval request
Facade
processFinancialApproval
Create ApprovalRequest (state
->PendingApproval)
Resolve approver
(RoutingSStrategy.determineAssignee)
Notify first approver
(ApprovalHandler)
Log submission
(AuditTrailObserver)
Start 48h timeout timer
(TimeoutEscalationObserver)
merge
[timeout >= 48h]
Set state ->Escalated
(EscalatedState
[response received]
Reassign to escalation approver
(ApprovalHandler.setNext)
Evaluate Request
[limit exceeded]
Forward request (next handler)
[approved]
[rejected]
merge
Notify observers, log outcome
(EventObserver.onWorkflowEvent)
Notify requester (Facade reports
outcome)

3.

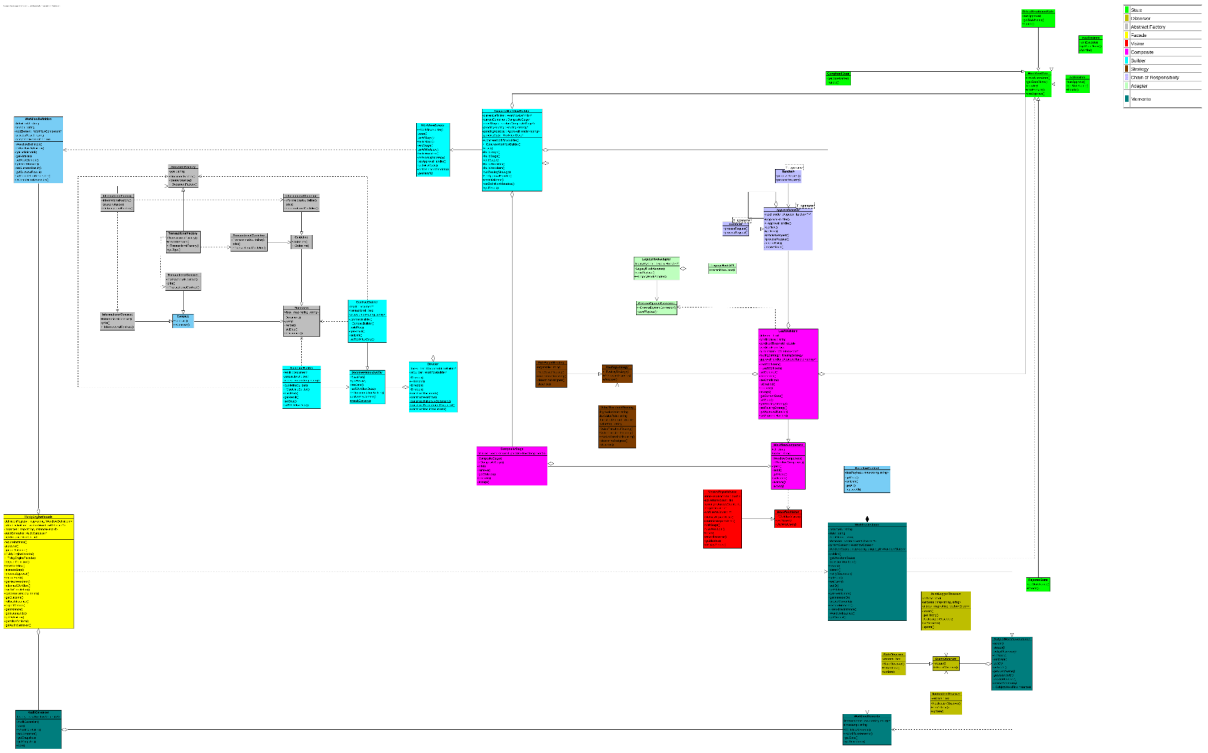

Client
Facade
WorkflowEngine
ProcurementRuleHandler
Approver
BudgetLedger
Submit material requisition (items, quantity, cost, justification)
Start workflow instance from procurement definition
Load workflow context and attach observers
Validate requisition data (completeness, format)
[invalid]
Correct and resubmit requisition
[valid]
Determine next work item and assign responsible role
Create and activate work item
Check construction specifications
Validate supplier (approved supplier list)
Check authority (approval limits)
Check funding (available budget)
[no]
Record rejection with rule failure reason(s)
Notify client of rejection and close instance
[yes]
Review requisition and supporting data
[reject]
Record rejection (with reviewer reason)
Notify client of rejection and close instance
[approve]
Record approval and update state (memento)
Generate purchase-order document
Commit budget (allocate funds)
Notify client of approval and provide document
Receive approval and purchase order (PO)

Task 4: GoF Design Patterns

Design Patterns	Participants
Builder	
Director	Director
Builder	WorkFlowBuilder, DocumentationBuilder
Concrete Builder	ConcreteWorkflowBuilder, GuidelineBuilder, ContractBuilder
Product	Document, WorkflowDefinition, Contract, Guideline
Observer	
Subject	SubjectWorkflowInstance
Concrete Subject	WorkflowInstance
Observer	EventObserver
Concrete Observer	NotificationObserver, StateObserver, AuditObserver
Strategy	
Strategy	RoutingStrategy
Concrete Strategy	ValueThresholdRouting, RoleBasedRouting
Context	LeafWorkItem
Chain Of Responsibility	
Handler	ApprovalHandler
Concrete Handler	HandlerA, HandlerB
State	
Concrete State	CompletedState, InactiveState, ActiveState, RejectedState, DefectResolutionState
State	WorkItemState
Visitor	
Visitor	WorkflowVisitor
Concrete Visitor	MetricsReportVisitor
Element	WorkflowComponent
Concrete Element	CompositeStage,LeafWorkItem

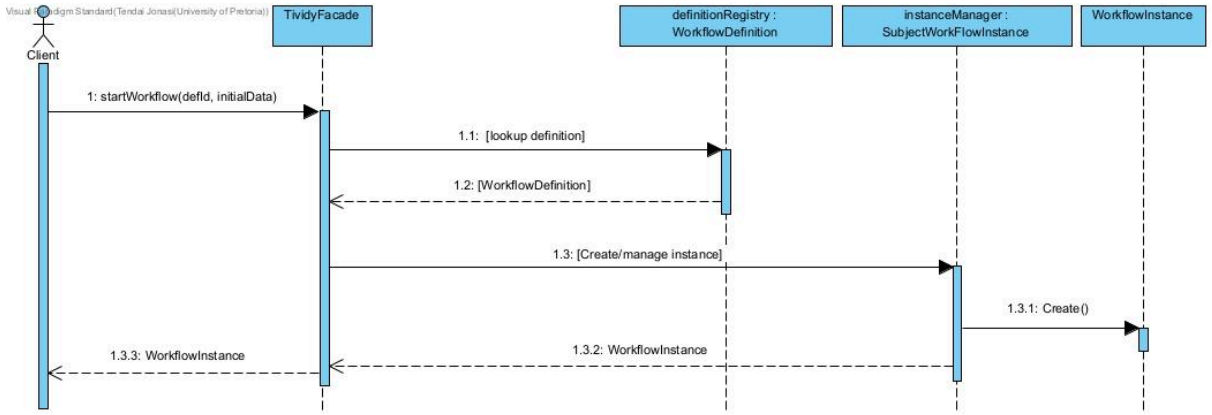
Object Structure	CompositeStage
Adapter	
Adaptee	LegacyiHookAPI
Target	ExternalSystemConnector
Adapter	LegacyiHookAdapter
Composite	
Component	WorkflowComponent
Composite	CompositeStage
Leaf	LeafWorkItem
Facade	TividyEngineFacade
Memento	
Originator	WorkflowInstance
Memento	WorkflowMemento
Caretaker	AuditCaretaker
Abstract Factory	
Abstract Factory	DocumentFactory
Concrete Factory	InformationalFactory, TransactionalFactory
Abstract Product	Contract, Guideline, Document
Concrete Product	InformationalContract, InformationalGuideline, TransactionalContract, TransactionalGuideline

Task 5: Class Diagram

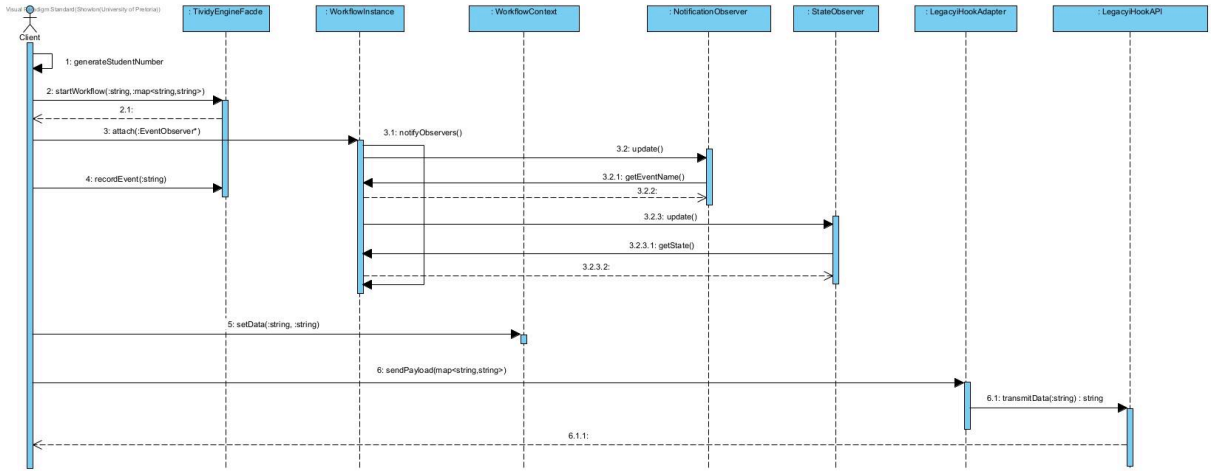


Task 6: Runtime Behaviour Diagrams

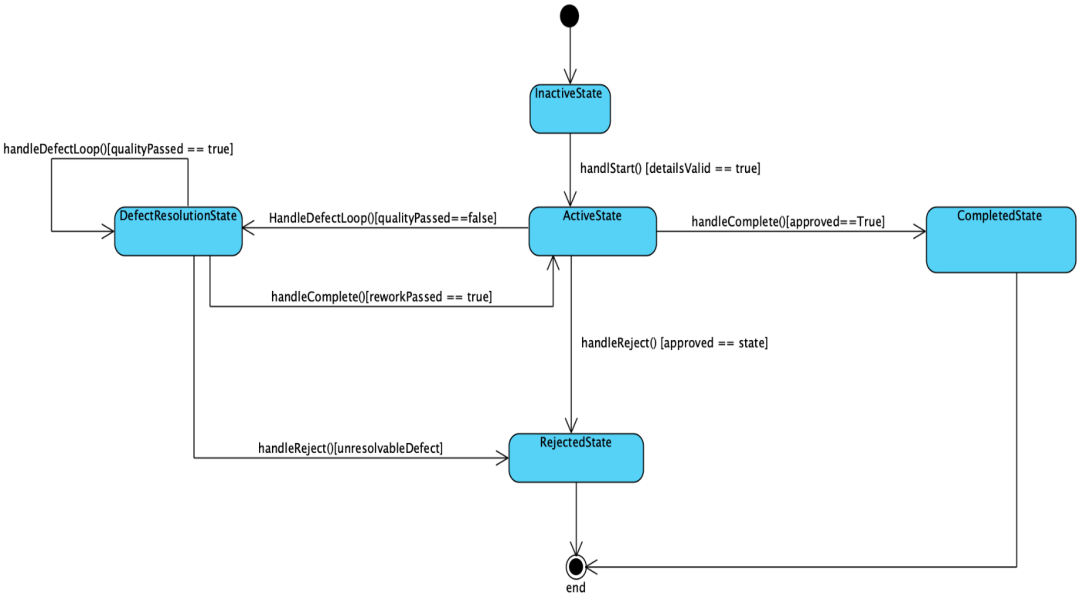
1. Sequence Diagram 1



2. Sequence Diagram 2



3. State diagram



Team Details

1. Lisakhanya Tatane u25514424
2. Tendai Jonasi u25010230
3. Siphiwe Aphane u24876829
4. Lethukuthula Ngwenya u24986535
5. Munashe Chitoro u24888096
6. Showlon Hunter u25502469
7. Shaun Mautsa u24693228

Github Repository Link

https://github.com/user6829/COS214-Prac6

Branching Process

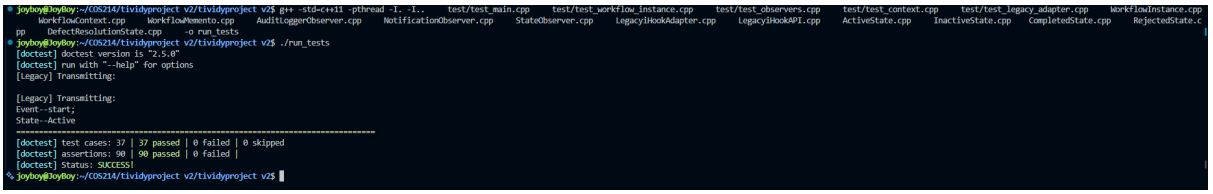
We first created a Development branch which branches off of the main branch. Here resides all the working files before they get committed to main. Then we have a UML-Diagrams branch to store our UML diagrams. A documents branch to store our final PDF report. Each person then has a feature branch which contains the files they are working on. Along with a test branch to conduct unit tests and system integration tests. Once the files in Development have been approved by everyone, they get committed to main. The main branch contains our final working product.

Automated Unit Tests:

State Design Pattern tests:

```
mwaba@DESKTOP-RAGP701:/mnt/c/Users/Dumisani/Documents/UP material/UP 2nd-year Modules/2n
ividyproject v2(tests)/tividyproject v2$ ./test_runner
[doctest] doctest version is "2.5.3"
[doctest] run with "--help" for options
Work item is Active.
Work item has been completed.
Work item is undergoing a defect resolution.
Work item is inactive.
Work item has been rejected.
===============================================================================
[doctest] test cases: 13 | 13 passed | 0 failed | 0 skipped
[doctest] assertions: 33 | 33 passed | 0 failed |
[doctest] Status: SUCCESS!
```

Observer & Adapter tests:



Composite Design Pattern tests:

```
===============================================================================
==
composite_tests.cpp:227:
TEST CASE:  WorkflowComponent* dispatch reaches overridden execute()

composite_tests.cpp:232: MESSAGE: dispatching execute() through base pointer:
 id = leaf, name = Leaf

composite_tests.cpp:240: MESSAGE: concrete executeCount = 1

===============================================================================
==
[doctest] test cases: 13 | 13 passed | 0 failed | 0 skipped
[doctest] assertions: 28 | 28 passed | 0 failed |
[doctest] Status: SUCCESS!
```

Builder and Abstract Factory Unit tests (doctests) :

```
./test_contract
[doctest] doctest version is "2.4.11"
[doctest] run with "--help" for options
=======================================
        INFORMATIONAL CONTRACT
=======================================
This document is provided for information
purposes only. It is non-binding and does
not create any legal obligations between
the parties involved.
---------------------------------------
Parties:   ____________________
Subject:   ____________________
Date:      ____________________
=======================================
=======================================
        TRANSACTIONAL CONTRACT
=======================================
This document constitutes a binding
agreement between the undersigned parties.
Both parties agree to the terms, conditions,
and obligations set forth herein.
---------------------------------------
Party A:   ____________________
Party B:   ____________________
Value:     ____________________
Term:      ____________________
Signature A: __________________
Signature B: __________________
=======================================
=======================================
       INFORMATIONAL GUIDELINE
=======================================
This guideline provides recommended
practices and general advice. Compliance
is optional; the content is advisory only.
---------------------------------------
Topic:     ____________________
Audience:  ____________________
Revision:  ____________________
=======================================
=======================================
       TRANSACTIONAL GUIDELINE
=======================================
This guideline describes mandatory
procedures that must be followed when
executing the associated transaction.
Non-compliance may void the transaction.
---------------------------------------
Procedure: ____________________
Authority: ____________________
Effective: ____________________
=======================================
===============================================================================
[doctest] test cases: 23 | 23 passed | 0 failed | 0 skipped
[doctest] assertions: 48 | 48 passed | 0 failed |
[doctest] Status: SUCCESS!
```

Chain of Responsibility & Memento Unit tests

```
siphiwe@Siphiwe:/mnt/c/Users/u24876829/Downloads/tividyproject v2/tividyproject v2 - Copy$ g++ -std=c++11 -g -fsanitize
o-omit-frame-pointer test.cpp ApprovalHandler.cpp HandlerA.cpp HandlerB.cpp AuditCaretaker.cpp WorkflowMemento.cpp Work
.cpp WorkflowContext.cpp ActiveState.cpp InactiveState.cpp CompletedState.cpp RejectedState.cpp DefectResolutionState.c
unner_asan && ./test_runner_asan
[doctest] doctest version is "2.5.3"
[doctest] run with "--help" for options
===============================================================================
[doctest] test cases:  5 |  5 passed | 0 failed | 0 skipped
[doctest] assertions: 37 | 37 passed | 0 failed |
[doctest] Status: SUCCESS!
```

Visitor and facade unit testing

```
g++ -std=c++11 -Wall -Wextra -g -I. -o tests/unit_tests doctest.o BudgetLedger.o WorkflowTemplateFactory.o WorkflowContext.o WorkflowComponent.o CompositeStage.o LeafWorkItem.o Workflow
finition.o WorkflowInstance.o WorkflowMemento.o AuditCaretaker.o ActiveState.o InactiveState.o CompletedState.o RejectedState.o DefectResolutionState.o RoutingStrategy.o RoleBasedRoutin
o ValueThresholdRouting.o ApprovalHandler.o HandlerA.o HandlerB.o UniversityRuleHandler.o ProcurementRuleHandler.o MetricsReportVisitor.o NotificationObserver.o StateObserver.o AuditLog
rObserver.o ConcreteWorkflowBuilder.o Director.o UniversityWorkflowDirector.o ProcurementWorkflowDirector.o InformationalContract.o InformationalGuideline.o TransactionalContract.o Tran
ctionalGuideline.o InformationalFactory.o TransactionalFactory.o ContractBuilder.o GuidelineBuilder.o LegacyiHookAPI.o LegacyiHookAdapter.o TividyEngineFacade.o -lncurses
./tests/unit_tests
[doctest] doctest version is "2.5.3"
[doctest] run with "--help" for options
===============================================================================
[doctest] test cases: 20 | 20 passed | 0 failed | 0 skipped
[doctest] assertions: 43 | 43 passed | 0 failed |
[doctest] Status: SUCCESS!
munashe@Munashes-Air tividyproject v2 %
```

Strategy Unit test

```
shaun@LAPTOP-0M245CUF:~/COS214/tividyproject v2 - Copy$ ./test
[doctest] doctest version is "2.5.3"
[doctest] run with "--help" for options
===============================================================================
[doctest] test cases: 23 | 23 passed | 0 failed | 0 skipped
[doctest] assertions: 24 | 24 passed | 0 failed |
[doctest] Status: SUCCESS!
shaun@LAPTOP-0M245CUF:~/COS214/tividyproject v2 - Copy$
```

Valgrind

```
==416368==    by 0x142012: main (main.cpp:2738)
==416368==
==416368== LEAK SUMMARY:
==416368==    definitely lost: 0 bytes in 0 blocks
==416368==    indirectly lost: 0 bytes in 0 blocks
==416368==      possibly lost: 0 bytes in 0 blocks
==416368==    still reachable: 157 bytes in 3 blocks
==416368==         suppressed: 0 bytes in 0 blocks
==416368==
==416368== For lists of detected and suppressed errors, rerun with: -s
==416368== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
siphiwe@Siphiwe:/tmp/tividy-valgrind$
```

GDB(More info in the txt file)

```
shaun@LAPTOP-0M245CUF:~/COS214/tividyproject v2/tividyproject v2$ g++ -std=c++11 -g -O0 -I. $(find . -maxdepth 1 -name '*.cpp' ! -name 'ge
ric_driver_reference.cpp' ! -name 'test.cpp') -o tividy_debug -lncurses
shaun@LAPTOP-0M245CUF:~/COS214/tividyproject v2/tividyproject v2$ ls -lh tividy_debug
-rwxr-xr-x 1 shaun shaun 7.8M Nov  1 23:24 tividy_debug
shaun@LAPTOP-0M245CUF:~/COS214/tividyproject v2/tividyproject v2$ gdb -q ./tividy_debug
Reading symbols from ./tividy_debug...
(gdb) set pagination off
(gdb) break main
Breakpoint 1 at 0xc8a49: file ./main.cpp, line 2720.
(gdb) break main.cpp:2742
Breakpoint 2 at 0xc8d33: file ./main.cpp, line 2742.
(gdb) info breakpoints
Num     Type           Disp Enb Address            What
1       breakpoint     keep y   0x0000000000c8a49 in main(int, char**) at ./main.cpp:2720
2       breakpoint     keep y   0x0000000000c8d33 in main(int, char**) at ./main.cpp:2742
(gdb) run
Starting program: /home/shaun/COS214/tividyproject v2/tividyproject v2/tividy_debug
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main (argc=1, argv=0x7fffffffda18) at ./main.cpp:2720
2720    {
(gdb) continue
```

Where changes were needed

- UML diagram notational errors
- UML Class diagram design
- Github Branching Strategy

Design Decision History

1. Generic workflow within a specialized domain i.e construction domain.
2. Generic engine that facilitates the operation of different mains.
3. Generic engine that facilitates the ability to define and run an arbitrary workflow at run-time, thus eliminating the act creating a new main for every new workflow scenario.
4. The scenarios chosen are a University Application Process and a Material Procurement Process without compromising the ability to define and run a new workflow at run-time.

Team Contribution Statement

We all took part in the planning of the execution of our project. We all conducted and compiled the research required for task 1. Then the Class diagram was done by equally partitioning the chosen design patterns amongst ourselves. The UML diagrams were also done by equally partitioning them. The implementation of the code was done by equally assigning the design patterns to everyone with the condition that we all help each other where necessary.